

УТВЕРЖДАЮ

Руководитель разработки

_____ к.т.н., доц. Машкин М. Н.

«____» _____ 2026

Программа расчёта
параметров эквипотенциальных
поверхностей

Вариант 2

Текст программы

Лист утверждения

643.02066606.00002-01 12 01-ЛУ

Инд. № подл.	Подпись и дата	Взам. инв. №	Инд. № дубл.	Подпись и дата

Исполнитель

Студент группы МЗО-225БВ-24

_____ Варфоломеев К. В.

«____» _____ 2026

УТВЕРЖДЕНО

643.02066606.00002-01 12 01-ЛУ

Программа расчёта
параметров эквипотенциальных
поверхностей

Вариант 2

Текст программы

643.02066606.00002-01 12 01

Листов 53

Инов. № подл.	Подпись и дата	Взам. инв. №	Инов. № дубл.	Подпись и дата

1 ТЕКСТ ПРОГРАММЫ VECTORFIELD НА ИСХОДНОМ ЯЗЫКЕ

1.1 Файл заголовков Resource.h

```

1  /*****
2  * КАФЕДРА № 304 | 2 КУРС | УЧЕБНАЯ ПРАКТИКА *
3  *-----*
4  * Project Name : Программа расчета параметров векторных полей *
5  * (Теория поля) *
6  * File Name : Resource.h *
7  * Author(s) : Варфоломеев К.В., Нагороднюк А.Д., *
8  * Великоцкая А.С. *
9  * Created : 02/07/2026 *
10 * Last Modified : 09/07/2026 *
11 * Version : 1.0 *
12 * Copyright : (c) 2026. All rights reserved. *
13 * License : Educational use only *
14 *-----*
15 * НАЗНАЧЕНИЕ ФАЙЛА: *
16 * Содержит макроопределения идентификаторов графических ресурсов и *
17 * элементов управления пользовательского интерфейса Win32. *
18 * *
19 * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ: *
20 * Файл Resource.h автоматически генерируется средой разработки *
21 * Microsoft Visual Studio и дополняется разработчиком для связывания *
22 * визуальных компонентов (кнопок, диалоговых окон, иконок) с их *
23 * числовыми идентификаторами в таблице ресурсов приложения. *
24 * *
25 * В ДАННОМ ФАЙЛЕ ОПРЕДЕЛЕНЫ: *
26 * 1. Системные идентификаторы приложения (IDS_APP_TITLE, *
27 * IDR_MAINFRAME, IDI_VECTORFIELD, IDI_SMALL). *
28 * 2. Идентификаторы диалоговых окон меню и подсистемы "0 программе" *
29 * (IDD_VECTORFIELD_DIALOG, IDD_ABOUTBOX). *
30 * 3. Уникальные идентификаторы управляющих кнопок на панели *
31 * инструментов, обеспечивающие обработку пользовательских событий *
32 * в оконной процедуре: *
33 * - ID_BTN_IMPORT : Импорт данных из внешнего файла .txt *
34 * - ID_BTN_SHOW : Отображение матрицы полиномов *
35 * - ID_BTN_CALC : Запуск процедур математического анализа полей *
36 * - ID_BTN_EXPORT : Экспорт сформированного отчета на диск *
37 *****/
38
39 //{NO_DEPENDENCIES}

```

```
40 // Включаемый файл, созданный в Microsoft Visual C++.
41 // Используется VectorField.rc
42 #define IDS_APP_TITLE          103
43 #define IDR_MAINFRAME          128
44 #define IDD_VECTORFIELD_DIALOG 102
45 #define IDD_ABOUTBOX           103
46 #define IDM_ABOUT              104
47 #define IDM_EXIT               105
48 #define IDI_VECTORFIELD        107
49 #define IDI_SMALL              108
50 #define IDC_VECTORFIELD        109
51 #define IDC_MYICON             2
52 #ifndef IDC_STATIC
53 #define IDC_STATIC             -1
54 #endif
55 // Кнопки интерфейса
56 #define ID_BTN_IMPORT          1001
57 #define ID_BTN_SHOW           1002
58 #define ID_BTN_CALC           1003
59 #define ID_BTN_VIS_A          1004
60 #define ID_BTN_VIS_B          1005
61 #define ID_BTN_EXPORT          1006
62 // Следующие стандартные значения для новых объектов
63 #ifdef APSTUDIO_INVOKED
64 #ifndef APSTUDIO_READONLY_SYMBOLS
65 #define _APS_NO_MFC             130
66 #define _APS_NEXT_RESOURCE_VALUE 129
67 #define _APS_NEXT_COMMAND_VALUE 32771
68 #define _APS_NEXT_CONTROL_VALUE 1007
69 #define _APS_NEXT_SYMED_VALUE  110
70 #endif
71 #endif
```

1.2 Файл заголовков VectorField.h

```

1  /*****
2  * КАФЕДРА № 304 | 2 КУРС | УЧЕБНАЯ ПРАКТИКА *
3  *-----*
4  * Project Name : Программа расчета параметров векторных полей *
5  * (Теория поля) *
6  * File Name : VectorField.h *
7  * Author(s) : Варфоломеев К.В., Нагороднюк А.Д., *
8  * Великоцкая А.С. *
9  * Created : 02/07/2026 *
10 * Last Modified : 09/07/2026 *
11 * Version : 1.0 *
12 * Copyright : (c) 2026. All rights reserved. *
13 * License : Educational use only *
14 *-----*
15 * НАЗНАЧЕНИЕ ФАЙЛА: *
16 * Содержит объявления базовых структур данных трехмерного пространства, *
17 * класса символьных полиномов и статических методов аналитического *
18 * ядра для вычислений в теории поля. *
19 * *
20 * ОПИСАНИЕ ИНТЕРФЕЙСОВ И ТИПОВ ДАННЫХ: *
21 * В заголовочном файле сосредоточены все типы данных, на которых *
22 * базируется вычислительная логика программы: *
23 * *
24 * 1. Структура Vector3D: *
25 * Описывает трехмерный геометрический вектор с вещественными *
26 * координатами (x, y, z). *
27 * *
28 * 2. Структура Monomial: *
29 * Описывает структуру отдельного монома, содержащего вещественный *
30 * коэффициент "coef" и целочисленные степени трех пространственных *
31 * переменных (px, py, pz). *
32 * *
33 * 3. Класс Polynomial: *
34 * Инкапсулирует логику символьной работы со многочленами (сложение, *
35 * зануление переменных при расчете потенциалов, символьное *
36 * интегрирование, численное вычисление значения в точке). *
37 * *
38 * 4. Класс FieldMathCalculations: *
39 * Математическое ядро программы, выполняющее синтаксический анализ *
40 * строковых формул, расчет дивергенции, ротора, потока (через теорему *
41 * Гаусса), циркуляции (через теорему Стокса), а также нахождение *
42 * потенциала векторного поля. *
43 *****/
44 #pragma once
45 #include <string>
46 #include <vector>

```

```

47  #include <functional>
48  #include "resource.h"
49  using namespace std;
50  // =====
51  // ОБЪЯВЛЕНИЯ СТРУКТУР И КЛАССОВ
52  // =====
53  struct Vector3D {double x, y, z;}; struct Monomial {double coef; int px, py, pz;};
54  class Polynomial {
55  public:
56      vector<Monomial> terms; Polynomial(initializer_list<Monomial> init);
57      Polynomial(); Polynomial SetVariableToZero(bool zeroX, bool zeroY,
58      bool zeroZ) const;
59      Polynomial IntegrateSymbolically(int varIndex) const;
60      void Add(const Polynomial& other); wstring ToWString() const;
61      double Evaluate(double x, double y, double z) const;
62  };
63  class FieldMathCalculations {
64  public: typedef function<double(double, double, double)> FUNC3D;
65      typedef function<Vector3D(double, double, double)> FUNC3D_VEC;
66      static Polynomial fieldA_X, fieldA_Y, fieldA_Z;
67      static Polynomial fieldB_X, fieldB_Y, fieldB_Z;
68      static void ParsePolynomials(const wstring& a_str, const wstring& b_str);
69      static Vector3D EvaluateFieldA(double x, double y, double z);
70      static Polynomial GetFieldBX(); static Polynomial GetFieldBY();
71      static Polynomial GetFieldBZ();
72      static Vector3D EvaluateFieldB(double x, double y, double z);
73      static double Calc_PartialDerivative(FUNC3D f, int variable,
74      double x, double y, double z);
75      static double IntegrateQuarterDisk(function<double(double, double)> func,
76      double radius = 1.0, int steps = 50);
77      static double Calc_Divergence(FUNC3D_VEC field, double x, double y, double z);
78      static Vector3D Calc_Rotor(FUNC3D_VEC field, double x, double y, double z);
79      static wstring DetermineFieldClass(FUNC3D_VEC field);
80      static double Calc_Flux_GaussOstrogradsky();
81      static double Calc_Flux_Direct(); static double Calc_Circulation_Stokes();
82      static double Calc_Circulation_Direct(); static Polynomial GetAnalyticalPotential();
83      static bool VerifyPotentialCorrectness(); static bool CheckRotorBZero();
84  private: static Polynomial ParsePoly(wstring expr);
85      static void ExtractComponents(wstring line, Polynomial& P,
86      Polynomial& Q, Polynomial& R);
87      static void ReplaceAll(wstring& str, const wstring& from, const wstring& to);
88  };

```

1.3 Метод Polynomial::SetVariableToZero

```

1  /*****
2  * КАФЕДРА № 304 | 2 КУРС | УЧЕБНАЯ ПРАКТИКА
3  *-----*
4  * Проект      : Программа расчета параметров векторных полей
5  * Файл        : VectorField.cpp
6  * Создан     : 02.07.2026
7  *
8  * РАСПРЕДЕЛЕНИЕ ОТВЕТСТВЕННОСТИ (ФУНКЦИОНАЛЬНЫЕ МОДУЛИ):
9  * - Великоцкая А.С. : Модули 1-20 (Математическое ядро)
10 * - Варфоломеев К.В. : Модули 20-35 (Вычисления и интерфейс)
11 * - Нагороднюк А.Д. : Модули 35-49 (Интерфейс и визуализация)
12 *-----*
13 * НАЗНАЧЕНИЕ ФАЙЛА:
14 * Реализация логики работы системы: описание полей, алгоритмы анализа,
15 * обработка данных и управление интерфейсом (WinAPI).
16 *****/
17 /*****
18 * Метод      : Polynomial::SetVariableToZero
19 *-----*
20 * Назначение : Символьное зануление переменных в полиноме
21 * Входные   : zeroX, zeroY, zeroZ - флаги зануления осей x, y, z
22 * Выходные  : Новый объект Polynomial с отфильтрованными мономами
23 *-----*
24 * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:
25 * Метод осуществляет символьное зануление выбранных переменных в
26 * структуре полинома. Это необходимо для подготовки к аналитическому
27 * интегрированию (потенциала поля). Метод фильтрует мономы, отбрасывая
28 * те, у которых степени зануляемых переменных больше нуля.
29 *****/
30 Polynomial Polynomial::SetVariableToZero(bool zeroX, bool zeroY,
31                                         bool zeroZ) const {
32     Polynomial result;
33     for (const auto& term : terms) {
34         if ((zeroX && term.px > 0) || (zeroY && term.py > 0) ||
35             (zeroZ && term.pz > 0)) continue;
36         result.terms.push_back(term);
37     }
38     return result;
39 }

```

1.4 Метод Polynomial::IntegrateSymbolically

```

1  /*****
2  * Метод      : Polynomial::IntegrateSymbolically      *
3  *-----*
4  * Назначение : Символьное интегрирование полинома по оси      *
5  * Входные   : varIndex - индекс переменной (0 -> x, 1 -> y, 2 -> z) *
6  * Выходные  : Новый объект Polynomial (первообразная)      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
9  * Выполняет аналитическое интегрирование полинома по выбранной оси.      *
10 * Метод обходит каждый моном и применяет правило интегрирования для      *
11 * степенной функции. Результат - новая первообразная функция,      *
12 * используемая для точного расчета потенциала.      *
13 *****/
14 Polynomial Polynomial::IntegrateSymbolically(int varIndex) const {
15     Polynomial result;
16     for (const auto& term : terms) {
17         Monomial m = term;
18         if (varIndex == 0) { m.px += 1; m.coef /= (double)m.px; }
19         else if (varIndex == 1) { m.py += 1; m.coef /= (double)m.py; }
20         else if (varIndex == 2) { m.pz += 1; m.coef /= (double)m.pz; }
21         result.terms.push_back(m);
22     }
23     return result;
24 }

```


1.5 Метод Polynomial::Add

```

1  /*****
2  * Метод          : Polynomial::Add                                *
3  *-----*
4  * Назначение     : Алгебраическое сложение полиномов          *
5  * Входные        : other - ссылка на складываемый полином      *
6  * Выходные       : Нет (модифицирует текущий объект)          *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                    *
9  * Реализует сложение многочленов с приведением подобных членов. *
10 * Алгоритм: 1. Поиск подобных слагаемых по набору степеней (px, py, pz). *
11 * 2. Очистка от нулевых членов с коэффициентом < 1e-6 (микропогрешности). *
12 *****/
13 void Polynomial::Add(const Polynomial& other) {
14     for (const auto& otherTerm : other.terms) {
15         bool found = false;
16         for (auto& myTerm : terms) {
17             if (myTerm.px == otherTerm.px && myTerm.py == otherTerm.py &&
18                 myTerm.pz == otherTerm.pz) {
19                 myTerm.coef += otherTerm.coef; found = true; break;
20             }
21         }
22         if (!found) terms.push_back(otherTerm);
23     }
24     terms.erase(remove_if(terms.begin(), terms.end(),
25         [](const Monomial& m) { return abs(m.coef) < 1e-6; }),
26         terms.end());
27 }

```

1.6 Метод Polynomial::Evaluate

```

1  /*****
2  * Метод          : Polynomial::Evaluate
3  *-----*
4  * Назначение     : Нахождение численного значения полинома в точке
5  * Входные        : x, y, z - координаты расчетной точки
6  * Выходные       : Вещественное число double - значение в точке
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:
9  * Производит численное вычисление значения многочлена в заданной точке
10 * пространства. Математическая модель: вычисление суммы произведений
11 * коэффициентов на переменные в степенях. Используется функция pow
12 * из заголовочного файла <cmath>.
13 *****/
14 double Polynomial::Evaluate(double x, double y, double z) const {
15     double sum = 0.0;
16     for (const auto& t : terms)
17         sum += t.coef * pow(x, t.px) * pow(y, t.py) * pow(z, t.pz);
18     return sum;
19 }

```

1.7 Метод Polynomial::ToWString

```

1  /*****
2  * Метод          : Polynomial::ToWString          *
3  *-----*
4  * Назначение     : Форматирование полинома в строковое представление *
5  * Входные       : Нет                               *
6  * Выходные      : wstring - строка широких символов (формат мономов) *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                      *
9  * Преобразует внутреннее представление полинома в строку вида        *
10 * "ax^2 + by - cz^3" для вывода в графическом интерфейсе.           *
11 * *
12 * АЛГОРИТМ:                                          *
13 * 1. Если список "terms" пуст, возвращается строковый ноль ("0").    *
14 * 2. Итеративно обходится каждый моном. Определяются знаки слагаемых. *
15 * 3. Форматируется вещественный коэффициент (std::setprecision(4)).    *
16 * 4. Единичные коэффициенты (при наличии переменных) опускаются.     *
17 * 5. Дописываются переменные x, y, z со степенями (если > 1).        *
18 *****/
19 wstring Polynomial::ToWString() const {
20     if (terms.empty()) return L"0";
21     wstringstream ss; bool first = true;
22     for (const auto& term : terms) {
23         if (term.coef > 0 && !first) ss << L" + ";
24         if (term.coef < 0) ss << (first ? L"-" : L" - ");
25         double absCoef = abs(term.coef);
26         if (absCoef != 1.0 || (term.px == 0 && term.py == 0 &&
27             term.pz == 0)) ss << setprecision(4) << absCoef;
28         if (term.px > 0) { ss << L"x"; if (term.px > 1) ss << L"^" << term.px; }
29         if (term.py > 0) { ss << L"y"; if (term.py > 1) ss << L"^" << term.py; }
30         if (term.pz > 0) { ss << L"z"; if (term.pz > 1) ss << L"^" << term.pz; }
31         first = false;
32     }
33     return ss.str();
34 }

```

1.8 Функция FieldMathCalculations::ReplaceAll

```

1  /*****
2  * Функция      : FieldMathCalculations::ReplaceAll      *
3  *-----*
4  * Назначение   : Глобальная замена всех вхождений подстроки в строке *
5  * Входные      : str - модифицируемая строка,          *
6  * from - искомая подстрока, to - заменяющая           *
7  * Выходные     : Нет (модифицирует строку по ссылке)   *
8  *-----*
9  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                             *
10 * Осуществляет поиск и замену всех вхождений подстроки "from" на      *
11 * значение "to" внутри строки "str".                               *
12 *****/
13 void FieldMathCalculations::ReplaceAll(wstring& str,
14                                       const wstring& from,
15                                       const wstring& to) {
16     if (from.empty()) return;
17     size_t start_pos = 0;
18     while ((start_pos = str.find(from, start_pos)) != wstring::npos) {
19         str.replace(start_pos, from.length(), to);
20         start_pos += to.length();
21     }
22 }

```

1.9 Функция FieldMathCalculations::ParsePoly

```

1  /*****
2  * Функция      : FieldMathCalculations::ParsePoly      *
3  *-----*
4  * Назначение   : Синтаксический анализ строкового полинома      *
5  * Входные     : expr - анализируемая строка формулы      *
6  * Выходные    : Объект Polynomial (структура мономов)      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
9  * Разбирает математическое выражение полинома от x, y, z.      *
10 * *
11 * АЛГОРИТМ:      *
12 * 1. Очистка строки от пробелов/скобок. Замена "-" на "+-".      *
13 * 2. Разделение строки на токены по "+".      *
14 * 3. Извлечение коэффициента и степеней переменных (x, y, z).      *
15 *****/
16 Polynomial FieldMathCalculations::ParsePoly(wstring expr) {
17     Polynomial poly; wstring clean;
18     for (wchar_t c : expr) {
19         if (c == L' ' || c == L'(' || c == L')' || c == L'*') continue;
20         if (c == L'-' ) { clean += L"+-"; continue; } clean += c; }
21     wstringstream ss(clean); wstring token;
22     while (getline(ss, token, L'+')) {
23         if (token.empty()) continue;
24         Monomial m = { 1.0, 0, 0, 0 };
25         size_t i = (token[0] == L'-' ) ? 1 : 0;
26         wstring coef = (token[0] == L'-' ) ? L"-": L"";
27         while (i < token.length() && (iswdigit(token[i]) ||
28             token[i] == L'.')) coef += token[i++];
29         m.coef = (coef == L"-": || coef.empty()) ?
30             ((coef == L"-") ? -1.0 : 1.0) : stod(coef);
31         auto pV = [&](wchar_t v, int& p) { size_t pos = token.find(v);
32             if (pos != wstring::npos) { p = 1;
33                 if (pos + 1 < token.length() && token[pos + 1] == L'^')
34                     p = stoi(token.substr(pos + 2)); }
35             };
36         pV(L'x', m.px); pV(L'y', m.py); pV(L'z', m.pz);
37         poly.terms.push_back(m);}
38     return poly;
39 }

```

1.10 Функция FieldMathCalculations::ExtractComponents

```

1  /*****
2  * Функция      : FieldMathCalculations::ExtractComponents      *
3  *-----*
4  * Назначение   : Выделение координатных функций P, Q, R      *
5  * Входные     : line - строка формулы, P, Q, R - ссылки на полиномы *
6  * Выходные    : Нет                                           *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                  *
9  * Извлекает полиномы P, Q, R из строки векторного поля.      *
10 * 1. Нормализация LaTeX-символики.                             *
11 * 2. Разбор строки при наличии ортов (i, j, k) или через разделитель ";". *
12 *****/
13 void FieldMathCalculations::ExtractComponents(wstring line, Polynomial& P,
14                                              Polynomial& Q, Polynomial& R) {
15     ReplaceAll(line, L"\\overline{i}", L"i");
16     ReplaceAll(line, L"\\overline{j}", L"j");
17     ReplaceAll(line, L"\\overline{k}", L"k");
18     ReplaceAll(line, L"\\overline{a}=", L"");
19     ReplaceAll(line, L"\\overline{b}=", L"");
20     size_t i_p = line.find(L"i"), j_p = line.find(L"j"), k_p = line.find(L"k");
21     if (i_p != wstring::npos && j_p != wstring::npos && k_p != wstring::npos) {
22         auto trimP = [](wstring s) {
23             size_t i = 0;
24             while (i < s.length() && (s[i] == L' ' || s[i] == L'+')) i++;
25             return s.substr(i); };
26         P = ParsePoly(line.substr(0, i_p));
27         Q = ParsePoly(trimP(line.substr(i_p + 1, j_p - i_p - 1)));
28         R = ParsePoly(trimP(line.substr(j_p + 1, k_p - j_p - 1)));}
29     else {
30         wstringstream ss(line); wstring item;
31         if (getline(ss, item, L';')) P = ParsePoly(item);
32         if (getline(ss, item, L';')) Q = ParsePoly(item);
33         if (getline(ss, item, L';')) R = ParsePoly(item);
34     }
35 }

```

1.11 Метод FieldMathCalculations::ParsePolynomials

```

1  /*****
2  * Метод          : FieldMathCalculations::ParsePolynomials      *
3  *-----*
4  * Назначение     : Парсинг строковых выражений в структуру полиномов *
5  * Входные       : a_str, b_str - строки формул полей A и B      *
6  * Выходные      : Нет (обновляет статические поля класса)      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                  *
9  * Метод преобразует текстовые представления векторных полей    *
10 * в математическую структуру Polynomial.                        *
11 * *
12 * АЛГОРИТМ:                                                    *
13 * 1. Вызывает ExtractComponents для каждой строки поля (A и B). *
14 * 2. ExtractComponents выполняет нормализацию и разделение на   *
15 * компоненты X, Y, Z.                                           *
16 * 3. Результаты сохраняются в статические переменные класса.   *
17 *****/
18 void FieldMathCalculations::ParsePolynomials(const wstring& a_str,
19                                               const wstring& b_str) {
20     ExtractComponents(a_str, fieldA_X, fieldA_Y, fieldA_Z);
21     ExtractComponents(b_str, fieldB_X, fieldB_Y, fieldB_Z);
22 }

```

1.12 Метод FieldMathCalculations::EvaluateFieldA

```

1  /*****
2  * Метод          : FieldMathCalculations::EvaluateFieldA          *
3  *-----*
4  * Назначение     : Вычисление вектора поля A в заданной точке     *
5  * Входные        : x, y, z - координаты точки                    *
6  * Выходные       : Vector3D - результирующий вектор поля в точке  *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Выполняет расчет численных значений компонент векторного поля A *
10 * в указанной точке.                                             *
11 * *
12 * АЛГОРИТМ:                                                       *
13 * 1. Обращается к компонентам fieldA_X, fieldA_Y, fieldA_Z.      *
14 * 2. Вызывает метод Evaluate для каждого компонента.             *
15 * 3. Объединяет результаты в структуру Vector3D.                 *
16 *****/
17 Vector3D FieldMathCalculations::EvaluateFieldA(double x, double y, double z) {
18     return { fieldA_X.Evaluate(x, y, z),
19             fieldA_Y.Evaluate(x, y, z),
20             fieldA_Z.Evaluate(x, y, z) };
21 }

```


1.13 Метод FieldMathCalculations::EvaluateFieldB

```

1  /*****
2  * Метод          : FieldMathCalculations::EvaluateFieldB          *
3  *-----*
4  * Назначение     : Вычисление вектора поля В в заданной точке     *
5  * Входные        : x, y, z - координаты точки                     *
6  * Выходные       : Vector3D - результирующий вектор поля в точке   *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Аналогичен EvaluateFieldA, но работает с полем В, используя методы- *
10 * геттеры для доступа к компонентам.                               *
11 * *
12 * АЛГОРИТМ:                                                       *
13 * 1. Вызывает методы GetFieldBX, GetFieldBY, GetFieldBZ.          *
14 * 2. Применяет метод Evaluate для каждого компонента.              *
15 * 3. Возвращает вектор в виде структуры Vector3D.                  *
16 *****/
17 Vector3D FieldMathCalculations::EvaluateFieldB(double x, double y, double z) {
18     return { GetFieldBX().Evaluate(x, y, z),
19             GetFieldBY().Evaluate(x, y, z),
20             GetFieldBZ().Evaluate(x, y, z) };
21 }

```

1.14 Метод FieldMathCalculations::Calc_PartialDerivative

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_PartialDerivative      *
3  *-----*
4  * Назначение     : Численное вычисление частной производной функции  *
5  * Входные        : f - функция, variable - индекс переменной, x,y,z   *
6  * Выходные       : double - приближенное значение производной         *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                         *
9  * Вычисляет производную функции f по переменной x, y или z          *
10 * методом центральной разности.                                       *
11 * *
12 * АЛГОРИТМ:                                                            *
13 * 1. Шаг дифференцирования h = 1e-4.                                   *
14 * 2. Вычисляет отношение: (f(x+h) - f(x-h)) / (2*h).                 *
15 *****/
16 double FieldMathCalculations::Calc_PartialDerivative(FUNC3D f, int variable,
17                                                         double x, double y,
18                                                         double z) {
19     double h = 1e-4;
20     if (variable == 1) return (f(x + h, y, z) - f(x - h, y, z)) / (2 * h);
21     if (variable == 2) return (f(x, y + h, z) - f(x, y - h, z)) / (2 * h);
22     return (f(x, y, z + h) - f(x, y, z - h)) / (2 * h);
23 }

```

1.15 Метод FieldMathCalculations::IntegrateQuarterDisk

```

1  /*****
2  * Метод          : FieldMathCalculations::IntegrateQuarterDisk      *
3  *-----*
4  * Назначение     : Численное интегрирование функции по четверти диска *
5  * Входные       : func - подынтегральная функция, radius, steps    *
6  * Выходные      : double - значение интеграла                       *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Численное интегрирование двумерной функции по области четверти   *
10 * диска в полярных координатах (r, phi).                            *
11 * *
12 * АЛГОРИТМ:                                                         *
13 * 1. Переход к полярным координатам:  $x = r \cos(\phi)$ ,  $y = r \sin(\phi)$ . "phi"- *
14 * обозначение полярного угла в системе полярных координат. *
15 * 2. Вычисление двойной суммы по сеткам r и phi с учетом якобиана (r). *
16 *****/
17 double FieldMathCalculations::IntegrateQuarterDisk(
18     function<double(double, double)> func, double radius, int steps) {
19     double dr = radius / steps, dphi = (M_PI / 2.0) / steps, sum = 0.0;
20     for (int i = 0; i < steps; i++) {
21         double r = (i + 0.5) * dr;
22         for (int j = 0; j < steps; j++) {
23             sum += func(r * cos((j + 0.5) * dphi),
24                 r * sin((j + 0.5) * dphi)) * r;
25         }
26     }
27     return sum * dr * dphi;
28 }

```

1.16 Метод FieldMathCalculations::Calc_Divergence

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_Divergence      *
3  *-----*
4  * Назначение     : Численное вычисление дивергенции векторного поля *
5  * Входные       : field - функция поля, x, y, z - координаты точки *
6  * Выходные      : double - значение дивергенции в точке *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ: *
9  * Вычисляет дивергенцию (div F) векторного поля, используя определение *
10 * через частные производные компонент: div F = dP/dx + dQ/dy + dR/dz. *
11 * *
12 * АЛГОРИТМ: *
13 * 1. Векторное поле раскладывается на скалярные функции P, Q, R. *
14 * 2. Для каждой компоненты вычисляется соответствующая частная *
15 * производная по своей переменной через метод Calc_PartialDerivative. *
16 * 3. Результаты суммируются для получения итогового скалярного значения. *
17 *****/
18 double FieldMathCalculations::Calc_Divergence(FUNC3D_VEC field, double x,
19 double y, double z) {
20     FUNC3D P = [&](double px, double py, double pz) {
21         return field(px, py, pz).x; };
22     FUNC3D Q = [&](double px, double py, double pz) {
23         return field(px, py, pz).y; };
24     FUNC3D R = [&](double px, double py, double pz) {
25         return field(px, py, pz).z; };
26     return Calc_PartialDerivative(P, 1, x, y, z) +
27         Calc_PartialDerivative(Q, 2, x, y, z) +
28         Calc_PartialDerivative(R, 3, x, y, z);
29 }

```

1.17 Метод FieldMathCalculations::Calc_Rotor

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_Rotor          *
3  *-----*
4  * Назначение     : Численное вычисление ротора (curl) векторного поля *
5  * Входные        : field - функция поля, x, y, z - координаты точки *
6  * Выходные       : Vector3D - вектор ротора в точке *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ: *
9  * Вычисляет вектор ротора (rot F) по формуле векторного произведения *
10 * оператора набла на вектор поля. *
11 * *
12 * АЛГОРИТМ: *
13 * 1. Метод вычисляет шесть частных производных компонент поля. *
14 * 2. Формирует компоненты вектора ротора: *
15 * x: dR/dy - dQ/dz; y: dP/dz - dR/dx; z: dQ/dx - dP/dy. *
16 *****/
17 Vector3D FieldMathCalculations::Calc_Rotor(FUNC3D_VEC field, double x,
18                                           double y, double z) {
19     FUNC3D P = [&](double px, double py, double pz) {
20         return field(px, py, pz).x; };
21     FUNC3D Q = [&](double px, double py, double pz) {
22         return field(px, py, pz).y; };
23     FUNC3D R = [&](double px, double py, double pz) {
24         return field(px, py, pz).z; };
25     return {
26         Calc_PartialDerivative(R, 2, x, y, z) - Calc_PartialDerivative(Q, 3, x, y, z),
27         Calc_PartialDerivative(P, 3, x, y, z) - Calc_PartialDerivative(R, 1, x, y, z),
28         Calc_PartialDerivative(Q, 1, x, y, z) - Calc_PartialDerivative(P, 2, x, y, z)
29     };
30 }

```

1.18 Метод FieldMathCalculations::DetermineFieldClass

```

1  /*****
2  * Метод          : FieldMathCalculations::DetermineFieldClass      *
3  *-----*
4  * Назначение     : Классификация типа векторного поля            *
5  * Входные        : field - функция поля                          *
6  * Выходные       : wstring - название класса поля                *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Определяет тип поля (потенциальное, соленоидальное, гармоническое или *
10 * общего вида) путем численной проверки свойств дивергенции и ротора.  *
11 *****/
12 wstring FieldMathCalculations::DetermineFieldClass(FUNC3D_VEC field) {
13     bool is_div = true, is_rot = true;
14     for (double x = 0.2; x <= 0.8; x += 0.3) {
15         for (double y = 0.2; y <= 0.8; y += 0.3) {
16             if (abs(Calc_Divergence(field, x, y, 0.5)) > 1e-3) is_div = false;
17             Vector3D r = Calc_Rotor(field, x, y, 0.5);
18             if (abs(r.x) > 1e-3 || abs(r.y) > 1e-3 || abs(r.z) > 1e-3)
19                 is_rot = false;
20         }
21     }
22     if (is_div && is_rot) return L"Гармоническое";
23     return (is_rot ? L"Потенциальное" :
24             (is_div ? L"Соленоидальное" : L"Общего вида"));
25 }

```

1.19 Метод FieldMathCalculations::Calc_Flux_GaussOstrogradsky

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_Flux_GaussOstrogradsky      *
3  *-----*
4  * Назначение     : Расчет потока через теорему Остроградского-Гаусса      *
5  * Входные        : Нет                                                    *
6  * Выходные       : double - значение потока                              *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                             *
9  * Реализует вычисление потока векторного поля через замкнутую поверхность *
10 * путем сведения поверхностного интеграла к объемному по теореме Гаусса. *
11 *****/
12 double FieldMathCalculations::Calc_Flux_GaussOstrogradsky() {
13     double Pc = 0.0; int st = 25;
14     double d_r = 1.0 / st, d_p = (M_PI / 2.0) / st, d_t = (M_PI / 2.0) / st;
15     for (int i = 0; i < st; ++i) {
16         for (int j = 0; j < st; ++j) {
17             for (int k = 0; k < st; ++k) {
18                 double r = (i + 0.5) * d_r, p = (j + 0.5) * d_p, t = (k + 0.5) * d_t;
19                 double x = r * cos(t) * cos(p), y = r * cos(t) * sin(p), z = r * sin(t);
20                 Pc += Calc_Divergence(EvaluateFieldA, x, y, z)
21                     * (r * r * cos(t)) * d_r * d_p * d_t;
22             }
23         }
24     }
25     double P_z0 = -IntegrateQuarterDisk([](double x, double y) {
26         return EvaluateFieldA(x, y, 0.0).z; });
27     double P_x0 = -IntegrateQuarterDisk([](double y, double z) {
28         return EvaluateFieldA(0.0, y, z).x; });
29     double P_y0 = -IntegrateQuarterDisk([](double x, double z) {
30         return EvaluateFieldA(x, 0.0, z).y; });
31     return Pc - (P_z0 + P_x0 + P_y0);
32 }

```

1.20 Метод FieldMathCalculations::Calc_Flux_Direct

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_Flux_Direct      *
3  *-----*
4  * Назначение     : Прямое вычисление потока векторного поля    *
5  * Входные       : Нет                                           *
6  * Выходные      : double - значение потока                     *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                   *
9  * Выполняет прямое интегрирование векторного поля по поверхности сферы *
10 * в первой октанте (четверть диска).                             *
11 *****/
12 double FieldMathCalculations::Calc_Flux_Direct() {
13     return IntegrateQuarterDisk([](double x, double y) {
14         double z = sqrt(fmax(0.0, 1.0 - x * x - y * y));
15         if (z < 1e-5) return 0.0;
16         Vector3D a = EvaluateFieldA(x, y, z);
17         return (a.x * x + a.y * y + a.z * z) / z;
18     });
19 }

```


1.21 Метод FieldMathCalculations::Calc_Circulation_Stokes

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_Circulation_Stokes      *
3  *-----*
4  * Назначение     : Расчет циркуляции векторного поля через теорему Стокса *
5  * Входные       : Нет                                                *
6  * Выходные      : double - значение циркуляции                      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                         *
9  * Вычисляет циркуляцию поля по контуру, используя теорему Стокса (переход*
10 * от криволинейного интеграла к интегралу от ротора по поверхности).  *
11 * *
12 * АЛГОРИТМ:                                                           *
13 * 1. Интегрирование ротора поля по поверхности через IntegrateQuarterDisk. *
14 * 2. Вычисление проекции вектора ротора на нормаль к поверхности.      *
15 *****/
16 double FieldMathCalculations::Calc_Circulation_Stokes() {
17     return IntegrateQuarterDisk([](double x, double y) {
18         double z = sqrt(fmax(0.0, 1.0 - x * x - y * y));
19         if (z < 1e-5) return 0.0;
20         Vector3D rot = Calc_Rotor(EvaluateFieldA, x, y, z);
21         return (rot.x * x + rot.y * y + rot.z * z) / z;
22     });
23 }

```

1.22 Метод FieldMathCalculations::Calc_Circulation_Direct

```

1  /*****
2  * Метод          : FieldMathCalculations::Calc_Circulation_Direct      *
3  *-----*
4  * Назначение     : Прямое вычисление циркуляции                      *
5  * Входные        : Нет                                              *
6  * Выходные       : double - значение циркуляции                     *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                         *
9  * Выполняет непосредственное интегрирование поля вдоль граничных    *
10 * отрезков в первой октанте (три стороны контура).                  *
11 * *
12 * АЛГОРИТМ:                                                           *
13 * 1. Параметризация трех сторон контура.                             *
14 * 2. Численное суммирование скалярного произведения поля на касательный *
15 * вектор (dl) с шагом dphi.                                          *
16 *****/
17 double FieldMathCalculations::Calc_Circulation_Direct() {
18     int steps = 100;
19     double dphi = (M_PI / 2.0) / steps, sum = 0.0;
20     for (int i = 0; i < steps; i++) {
21         double phi = (i + 0.5) * dphi;
22         Vector3D a1 = EvaluateFieldA(cos(phi), sin(phi), 0.0);
23         sum += (a1.x * (-sin(phi)) + a1.y * cos(phi));
24         Vector3D a2 = EvaluateFieldA(0.0, cos(phi), sin(phi));
25         sum += (a2.y * (-sin(phi)) + a2.z * cos(phi));
26         Vector3D a3 = EvaluateFieldA(sin(phi), 0.0, cos(phi));
27         sum += (a3.z * (-sin(phi)) + a3.x * cos(phi));
28     }
29     return sum * dphi;
30 }

```

1.23 Метод FieldMathCalculations::GetAnalyticalPotential

```

1  /*****
2  * Метод          : FieldMathCalculations::GetAnalyticalPotential      *
3  *-----*
4  * Назначение     : Аналитическое нахождение потенциала поля В      *
5  * Входные        : Нет                                              *
6  * Выходные       : Polynomial - результирующая функция потенциала   *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                         *
9  * Вычисляет скалярный потенциал U(x,y,z), такой что grad(U) = B.    *
10 * *
11 * АЛГОРИТМ:                                                            *
12 * 1. Интегрирование компонент (Вх по x, Vy по y, Vz по z).          *
13 * 2. Использование метода SetVariableToZero для исключения перекрестных *
14 * членов, чтобы избежать дублирования данных при восстановлении.     *
15 * 3. Алгебраическое сложение полученных функций (Add).              *
16 *****/
17 Polynomial FieldMathCalculations::GetAnalyticalPotential() {
18     Polynomial U1 = GetFieldBX().SetVariableToZero(false, true, true)
19                     .IntegrateSymbolically(0);
20     Polynomial U2 = GetFieldBY().SetVariableToZero(false, false, true)
21                     .IntegrateSymbolically(1);
22     Polynomial U3 = GetFieldBZ().SetVariableToZero(false, false, false)
23                     .IntegrateSymbolically(2);
24     Polynomial U; U.Add(U1); U.Add(U2); U.Add(U3);
25     return U;
26 }

```

1.24 Метод FieldMathCalculations::VerifyPotentialCorrectness

```

1  /*****
2  * Метод          : FieldMathCalculations::VerifyPotentialCorrectness      *
3  *-----*
4  * Назначение     : Верификация точности найденного потенциала          *
5  * Входные       : Нет                                                    *
6  * Выходные      : bool - статус корректности                          *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                          *
9  * Проверяет условие  $\text{grad}(U) = B$  в контрольных точках сетки.      *
10 * *
11 * АЛГОРИТМ:                                                            *
12 * 1. В контрольных точках вычисляются частные производные от U.      *
13 * 2. Сравнение градиента с вектором поля B. Если разница превышает    *
14 * порог (0.05) в любой точке, возвращается false.                    *
15 *****/
16 bool FieldMathCalculations::VerifyPotentialCorrectness() {
17     Polynomial U = GetAnalyticalPotential();
18     FUNC3D U_eval = [&](double px, double py, double pz) {
19         return U.Evaluate(px, py, pz);
20     };
21     for (double x = 0.2; x <= 0.8; x += 0.3) {
22         for (double y = 0.2; y <= 0.8; y += 0.3) {
23             double du_dx = Calc_PartialDerivative(U_eval, 1, x, y, 0.5);
24             double du_dy = Calc_PartialDerivative(U_eval, 2, x, y, 0.5);
25             Vector3D valB = EvaluateFieldB(x, y, 0.5);
26             if (abs(du_dx - valB.x) > 0.05 || abs(du_dy - valB.y) > 0.05)
27                 return false;
28         }
29     }
30     return true;
31 }

```

1.25 Метод FieldMathCalculations::CheckRotorBZero

```

1  /*****
2  * Метод          : FieldMathCalculations::CheckRotorBZero          *
3  *-----*
4  * Назначение     : Проверка потенциальности поля В (rot В = 0)    *
5  * Входные        : Нет                                             *
6  * Выходные       : bool - результат проверки                       *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Численная проверка условия отсутствия вихрей в поле В.         *
10 * *
11 * АЛГОРИТМ:                                                       *
12 * 1. Сканирование области по сетке (x, y).                        *
13 * 2. Вычисление ротора в каждой точке.                             *
14 * 3. Если модуль ротора превышает 1e-3, поле считается непотенциальным. *
15 *****/
16 bool FieldMathCalculations::CheckRotorBZero() {
17     for (double x = 0.2; x <= 0.8; x += 0.3) {
18         for (double y = 0.2; y <= 0.8; y += 0.3) {
19             Vector3D r = Calc_Rotor(EvaluateFieldB, x, y, 0.5);
20             if (abs(r.x) > 1e-3 || abs(r.y) > 1e-3 || abs(r.z) > 1e-3)
21                 return false;
22         }
23     }
24     return true;
25 }

```

1.26 Функция GetLoadFileNameExplorer

```

1  /*****
2  * Функция      : GetLoadFileNameExplorer      *
3  *-----*
4  * Назначение   : Диалог выбора файла для открытия      *
5  * Входные     : hWnd - дескриптор окна, outPath - путь к файлу      *
6  * Выходные    : bool - успех операции      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
9  * Обертка над Win32 API для вызова стандартного диалога открытия файла. *
10 *****/
11 bool GetLoadFileNameExplorer(HWND hWnd, wstring& outPath) {
12     wchar_t filename[MAX_PATH] = L"";
13     OPENFILENAMEW ofn;
14     ZeroMemory(&ofn, sizeof(ofn));
15     ofn.lStructSize = sizeof(ofn);
16     ofn.hwndOwner = hWnd;
17     ofn.lpstrFilter = L"Текстовые файлы (*.txt)\0*.txt\0Все файлы (*.*)\0*.*\0";
18     ofn.lpstrFile = filename;
19     ofn.nMaxFile = MAX_PATH;
20     ofn.Flags = OFN_FILEMUSTEXIST | OFN_PATHMUSTEXIST;
21     if (GetOpenFileNameW(&ofn)) {
22         outPath = filename;
23         return true;
24     }
25     return false;
26 }

```

1.27 Функция GetSaveFileNameExplorer

```
1  /*****
2  * Функция      : GetSaveFileNameExplorer      *
3  *-----*
4  * Назначение   : Диалог выбора файла для сохранения      *
5  * Входные     : hWnd - дескриптор окна, outPath - путь к файлу      *
6  * Выходные    : bool - успех операции      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
9  * Обертка над Win32 API для вызова стандартного диалога сохранения файла. *
10 *****/
11 bool GetSaveFileNameExplorer(HWND hWnd, wstring& outPath) {
12     wchar_t filename[MAX_PATH] = L"output.txt";
13     OPENFILENAMEW ofn;
14     ZeroMemory(&ofn, sizeof(ofn));
15     ofn.lStructSize = sizeof(ofn);
16     ofn.hwndOwner = hWnd;
17     ofn.lpstrFilter = L"Текстовые файлы (*.txt)\0*.txt\0";
18     ofn.lpstrFile = filename;
19     ofn.nMaxFile = MAX_PATH;
20     ofn.Flags = OFN_OVERWRITEPROMPT | OFN_PATHMUSTEXIST;
21     if (GetSaveFileNameW(&ofn)) {
22         outPath = filename;
23         return true;
24     }
25     return false;
26 }
```

1.28 Функция ImportDataFromFile

```

1  /*****
2  * Функция      : ImportDataFromFile                                *
3  *-----*
4  * Назначение   : Импорт и парсинг данных из текстового файла      *
5  * Входные     : filename - путь к файлу                            *
6  * Выходные    : bool - успех импорта                               *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Читает вариант задания и формулы полей, выполняет их первичный парсинг. *
10 *****/
11 bool ImportDataFromFile(const wstring& filename) {
12     ifstream file(filename);
13     if (!file.is_open()) return false;
14     string line;
15     if (getline(file, line)) {
16         try { g_VariantNum = stoi(line); }
17         catch (...) { return false; }
18     }
19     if (getline(file, line))
20         g_FieldAStr = wstring(line.begin(), line.end());
21     if (getline(file, line))
22         g_FieldBStr = wstring(line.begin(), line.end());
23     file.close();
24     if (g_VariantNum <= 0 || g_FieldAStr.empty() || g_FieldBStr.empty())
25         return false;
26     FieldMathCalculations::ParsePolynomials(g_FieldAStr, g_FieldBStr);
27     return true;
28 }

```


1.29 Функция ExportDataToFile

```

1  /*****
2  * Функция      : ExportDataToFile                                *
3  *-----*
4  * Назначение   : Экспорт отчета в файл (UTF-8 с BOM)            *
5  * Входные     : filename - путь к файлу                        *
6  * Выходные    : bool - успех записи                             *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                    *
9  * Конвертирует отчет в UTF-8 и записывает в файл с добавлением BOM-метки. *
10 *****/
11 bool ExportDataToFile(const wstring& filename) {
12     wstring report = BuildReportText();
13     if (report.empty() || g_VariantNum == 0) return false;
14     int s_n = WideCharToMultiByte(CP_UTF8, 0, &report[0],
15                                   (int)report.size(), NULL, 0, NULL, NULL);
16     string utf8_str(s_n, 0);
17     WideCharToMultiByte(CP_UTF8, 0, &report[0], (int)report.size(),
18                         &utf8_str[0], s_n, NULL, NULL);
19     ofstream file(filename, ios::binary);
20     if (!file.is_open()) return false;
21     const unsigned char bom[] = { 0xEF, 0xBB, 0xBF };
22     file.write((const char*)bom, sizeof(bom));
23     file.write(utf8_str.c_str(), utf8_str.size());
24     file.close();
25     return true;
26 }

```

1.30 Функция BuildInputDataText

```

1  /*****
2  * Функция      : BuildInputDataText                                *
3  *-----*
4  * Назначение   : Формирование текстового блока исходных данных      *
5  * Входные      : Нет                                                *
6  * Выходные     : wstring - отформатированный текст                  *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                        *
9  * Создает текстовое представление входных данных для отображения в GUI. *
10 *****/
11 wstring BuildInputDataText() {
12     wstringstream ss;
13     ss << L"=====\\n";
14     ss << L"          ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА\\n";
15     ss << L"=====\\n\\n";
16     ss << L"   Загружен вариант №:  " << g_VariantNum << L"\\n\\n";
17     ss << L"   Векторное поле  $\bar{a}$  (исследуется на поток и циркуляцию):\\n";
18     ss << L"    $\bar{a}$  = " << g_FieldAStr << L"\\n\\n";
19     ss << L"   Векторное поле b (исследуется на потенциальность):\\n";
20     ss << L"   b = " << g_FieldBStr << L"\\n\\n";
21     ss << L"-----\\n";
22     ss << L"   Всё готово! Нажмите кнопку «Расчёт данных» для получения отчета.";
23     return ss.str();
24 }

```

1.31 Функция BuildReportText

```

1  /*****
2  * Функция      : BuildReportText                                     *
3  *-----*
4  * Назначение   : Формирование полного отчета по расчетам          *
5  * Входные      : Нет                                              *
6  * Выходные     : wstring - текст итогового отчета                 *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Собирает результаты вычислений (потoki, циркуляции, потенциал) *
10 * в единую строку для вывода в интерфейс или сохранения в файл.  *
11 *****/
12 wstring BuildReportText() { wstringstream rep; rep << fixed << setprecision(4);
13     Polynomial U = FieldMathCalculations::GetAnalyticalPotential();
14     rep << L"===== РАССЧИТАННЫЕ ДАННЫЕ (ВАР. №" << g_VariantNum << L") =====\n\n"
15         << L" 1. КЛАССИФИКАЦИЯ ПОЛЯ ā:\n    • Тип поля: "
16         << FieldMathCalculations::DetermineFieldClass
17         ( FieldMathCalculations::EvaluateFieldA) << L"\n\n"
18         << L" 2. ВЫЧИСЛЕНИЕ ПОТОКА ПОВЕРХНОСТИ S (ПОЛЕ ā):\n"
19         << L"    • 2.1 Метод Остроградского-Гаусса: P = "
20         << FieldMathCalculations::Calc_Flux_GaussOstrogradsky() << L"\n"
21         << L"    • 2.2 Метод прямого интегрирования: P = "
22         << FieldMathCalculations::Calc_Flux_Direct() << L"\n\n"
23         << L" 3. ВЫЧИСЛЕНИЕ ЦИРКУЛЯЦИИ ПО КОНТУРУ S (ПОЛЕ ā):\n"
24         << L"    • 3.1 Метод теоремы Стокса: C = "
25         << FieldMathCalculations::Calc_Circulation_Stokes() << L"\n"
26         << L"    • 3.2 Метод криволинейного интеграл: C = "
27         << FieldMathCalculations::Calc_Circulation_Direct() << L"\n\n"
28         << L" 4. ИССЛЕДОВАНИЕ ПОЛЯ b:\n" << L"    • 4.1 Проверка потенциальности: "
29         << (FieldMathCalculations::CheckRotorBZero() ?
30             L"поле потенциально" : L"поле не потенциально") << L"\n"
31         << L"    • 4.2 Потенциал U(x, y, z) = " << U.ToWString() << L"\n"
32         << L"U(1, 1, 1) = " << U.Evaluate(1, 1, 1) << L"\n" << L"    • 4.3 Проверка градиента: "
33         << (FieldMathCalculations::VerifyPotentialCorrectness() ?
34             L"успешна" : L"не успешна") << L"\n" << L"    • 4.4 Работа сил поля W = "
35         << U.Evaluate(1, 1, 1) - U.Evaluate(0, 0, 0) << L"\n\n"
36         << L"====="; return rep.str(); }

```

1.32 Процедура CreateInterfaceButtons

```

1  /*****
2  * Процедура      : CreateInterfaceButtons                      *
3  *-----*
4  * Назначение     : Создание кнопок управления интерфейсом    *
5  * Входные        : hWnd - окно, hInstance - экземпляр приложения *
6  * Выходные       : Нет                                         *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                  *
9  * Инициализирует элементы управления (кнопки) в рабочей области окна. *
10 *****/
11 void CreateInterfaceButtons(HWND hWnd, HINSTANCE hInstance) {
12     int y1 = 45, w = 185, h = 35, step = 190;
13     CreateWindowW(L"BUTTON", L"1. Импорт данных", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
14                 20 + 0 * step, y1, w, h, hWnd, (HMENU)ID_BTN_IMPORT,
15                 hInstance, NULL);
16     CreateWindowW(L"BUTTON", L"2. Показать данные", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
17                 20 + 1 * step, y1, w, h, hWnd, (HMENU)ID_BTN_SHOW,
18                 hInstance, NULL);
19     CreateWindowW(L"BUTTON", L"3. Расчёт данных", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
20                 20 + 2 * step, y1, w, h, hWnd, (HMENU)ID_BTN_CALC,
21                 hInstance, NULL);
22     CreateWindowW(L"BUTTON", L"4. Визуализация Поля А", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
23                 20 + 3 * step, y1, w, h, hWnd, (HMENU)ID_BTN_VIS_A,
24                 hInstance, NULL);
25     CreateWindowW(L"BUTTON", L"5. Визуализация Поля Б", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
26                 20 + 4 * step, y1, w, h, hWnd, (HMENU)ID_BTN_VIS_B, hInstance, NULL);
27     CreateWindowW(L"BUTTON", L"6. Экспорт в TXT", WS_VISIBLE | WS_CHILD | BS_PUSHBUTTON,
28                 20 + 5 * step, y1, w, h, hWnd, (HMENU)ID_BTN_EXPORT, hInstance, NULL);
29 }

```

1.33 Процедура DrawBackground

```
1  /*****
2  * Процедура      : DrawBackground      *
3  *-----*
4  * Назначение     : Отрисовка фона и информации о разработчиках      *
5  * Входные        : hdc - контекст устройства, rect - область отрисовки *
6  * Выходные       : Нет                                                    *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
9  * Заливает фон цветом и выводит строку с информацией о разработчиках. *
10 *****/
11 void DrawBackground(HDC hdc, RECT rect) {
12     HBRUSH hBgBrush = CreateSolidBrush(RGB(240, 244, 248));
13     FillRect(hdc, &rect, hBgBrush);
14     DeleteObject(hBgBrush);
15     HFONT hFontDev = CreateFontW(18, 0, 0, 0, FW_SEMIBOLD, 0, 0, 0, DEFAULT_CHARSET,
16         OUT_OUTLINE_PRECIS, CLIP_DEFAULT_PRECIS,
17         CLEARTYPE_QUALITY, VARIABLE_PITCH, L"Segoe UI");
18     HFONT hOld = (HFONT)SelectObject(hdc, hFontDev);
19     SetBkMode(hdc, TRANSPARENT); SetTextColor(hdc, RGB(0, 0, 0));
20     wstring devInfo = L"Разработчики: Варфоломеев, Великоцкая, Нагороднюк "
21         L" | Группа: М30-225БВ-24";
22     RECT devRect = { 25, 10, rect.right - 20, 35 };
23     DrawTextW(hdc, devInfo.c_str(), -1, &devRect,
24         DT_LEFT | DT_VCENTER | DT_SINGLELINE);
25     SelectObject(hdc, hOld); DeleteObject(hFontDev);
26 }
```

1.34 Процедура DrawStatusAndCard

```

1  /*****
2  * Процедура      : DrawStatusAndCard                      *
3  *-----*
4  * Назначение     : Отрисовка панели статуса и центральной "карточки" *
5  * Входные       : hdc - контекст устройства, rect - область отрисовки *
6  * Выходные      : Нет                                         *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                               *
9  * Рисует полосу статуса и прямоугольник (карточку) для вывода контента. *
10 *****/
11 void DrawStatusAndCard(HDC hdc, RECT rect) {
12     RECT sRect = { 20, 100, rect.right - 20, 130 };
13     HBRUSH hStatB = CreateSolidBrush(RGB(225, 235, 245));
14     FillRect(hdc, &sRect, hStatB); DeleteObject(hStatB);
15     HFONT hFontS = CreateFontW(16, 0, 0, 0, FW_BOLD, 0, 0, 0, DEFAULT_CHARSET,
16         OUT_OUTLINE_PRECIS, CLIP_DEFAULT_PRECIS,
17         CLEARTEXTYPE_QUALITY, VARIABLE_PITCH, L"Segoe UI");
18     HFONT hOld = (HFONT)SelectObject(hdc, hFontS);
19     SetTextColor(hdc, RGB(0, 0, 0));
20     RECT sText = { 35, 105, rect.right - 35, 125 };
21     DrawTextW(hdc, g_StatusMessage.c_str(), -1, &sText,
22         DT_LEFT | DT_VCENTER | DT_SINGLELINE);
23     RECT cR = { 20, 145, rect.right - 20, rect.bottom - 20 };
24     RECT shR = { cR.left + 3, cR.top + 3, cR.right + 3, cR.bottom + 3 };
25     HBRUSH hShB = CreateSolidBrush(RGB(220, 225, 230));
26     FillRect(hdc, &shR, hShB); DeleteObject(hShB);
27     HBRUSH hCardB = CreateSolidBrush(RGB(255, 255, 255));
28     HPEN hCardP = CreatePen(PS_SOLID, 1, RGB(200, 205, 215));
29     HPEN hOldP = (HPEN)SelectObject(hdc, hCardP);
30     HBRUSH hOldB = (HBRUSH)SelectObject(hdc, hCardB);
31     Rectangle(hdc, cR.left, cR.top, cR.right, cR.bottom);
32     SelectObject(hdc, hOldB); SelectObject(hdc, hOldP);
33     DeleteObject(hCardB); DeleteObject(hCardP);
34     SelectObject(hdc, hOld); DeleteObject(hFontS);
35 }

```

1.35 Процедура HandlePaint

```

1  /*****
2  * Процедура      : HandlePaint                                     *
3  *-----*
4  * Назначение     : Обработчик события перерисовки окна (WM_PAINT) *
5  * Входные       : hWnd - дескриптор окна                         *
6  * Выходные      : Нет                                           *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Вызывает функции отрисовки интерфейса и выводит контент в зависимости *
10 * от выбранного пользователем режима (вид 0-4).                  *
11 *****/
12 void HandlePaint(HWND hWnd) {
13     PAINTSTRUCT ps; HDC hdc = BeginPaint(hWnd, &ps); RECT r; GetClientRect(hWnd, &r);
14     DrawBackground(hdc, r); DrawStatusAndCard(hdc, r);
15     HFONT hF = CreateFontW(20, 0, 0, 0, FW_NORMAL, 0, 0, 0, DEFAULT_CHARSET,
16         OUT_OUTLINE_PRECIS, CLIP_DEFAULT_PRECIS, CLEAR_TYPE_QUALITY,
17         VARIABLE_PITCH, L"Segoe UI");
18     HFONT hOld = (HFONT)SelectObject(hdc, hF); SetTextColor(hdc, RGB(0, 0, 0));
19     RECT tR = { 40, 165, r.right - 40, r.bottom - 40 };
20     if (g_ActiveView == 1) { wstring t = BuildInputDataText();
21         DrawTextW(hdc, t.c_str(), -1, &tR, DT_LEFT | DT_WORDBREAK);}
22     else if (g_ActiveView == 2) { wstring t = BuildReportText();
23         DrawTextW(hdc, t.c_str(), -1, &tR, DT_LEFT | DT_WORDBREAK);}
24     else if (g_ActiveView == 3 || g_ActiveView == 4) DrawVisualization(hdc, tR);
25     else if (g_ActiveView == 0) { wstring t = L"\n\n ПРОГРАММА РАСЧЕТА ПАРАМЕТРОВ "
26         L"ВЕКТОРНЫХ ПОЛЕЙ\n -----"
27         L"-----\n\n"
28         L"      Для работы с программой:\n\n"
29         L"      1. Нажмите кнопку «1. Импорт данных» сверху\n"
30         L"      2. Выберите .txt файл с вариантом задания\n"
31         L"      3. Нажмите «3. Расчёт данных» для отчета\n"
32         L"      4. Используйте кнопки визуализации.\n"
33         L"      5. Сохраните результат (кнопка 6).\n";
34         DrawTextW(hdc, t.c_str(), -1, &tR, DT_LEFT | DT_WORDBREAK);}
35     SelectObject(hdc, hOld); DeleteObject(hF); EndPaint(hWnd, &ps);
36 }

```

1.36 Процедура HandleCmdImportShow

```

1  /*****
2  * Процедура      : HandleCmdImportShow                      *
3  *-----*
4  * Назначение     : Обработка команд импорта и отображения данных *
5  * Входные       : hWnd - дескриптор окна, wParam - идентификатор команды *
6  * Выходные      : Нет                                         *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                *
9  * Обработывает нажатия кнопок импорта и показа данных, управляет *
10 * состоянием интерфейса и сбросом предыдущих результатов.      *
11 *****/
12 void HandleCmdImportShow(HWND hWnd, int wParam) {
13     if (wParam == ID_BTN_IMPORT) { wstring f;
14         if (GetLoadFileNameExplorer(hWnd, f)) {
15             if (ImportDataFromFile(f)) {
16                 g_StatusMessage = L" Успех! Вариант №" + to_wstring(g_VariantNum);
17                 g_ActiveView = 1; g_IsCalculated = false;
18                 if (g_hPlotBitmapA) { DeleteObject(g_hPlotBitmapA); g_hPlotBitmapA = NULL;}
19                 if (g_hPlotBitmapB) { DeleteObject(g_hPlotBitmapB);
20                     g_hPlotBitmapB = NULL; }
21             } else ShowError(hWnd, ErrParse);}
22     }
23     else if (wParam == ID_BTN_SHOW) { if (g_VariantNum == 0) ShowError(hWnd, WarnNoData);
24         else { g_StatusMessage = L" Исходные данные ВАР №" + to_wstring(g_VariantNum);
25             g_ActiveView = 1; }
26     }
27 }

```


1.37 Процедура HandleCmdCalcExport

```

1  /*****
2  * Процедура      : HandleCmdCalcExport      *
3  *-----*
4  * Назначение     : Обработка команд расчета и экспорта      *
5  * Входные        : hWnd - дескриптор окна, wParam - идентификатор команды *
6  * Выходные       : Нет      *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
9  * Выполняет расчет параметров поля и экспорт отчета в файл.      *
10 *****/
11 void HandleCmdCalcExport(HWND hWnd, int wParam) {
12     if (wParam == ID_BTN_CALC) {
13         if (g_VariantNum == 0) ShowError(hWnd, WarnNoData);
14         else { g_StatusMessage = L" Расчёт выполнен!"; g_ActiveView = 2;
15             g_IsCalculated = true;}
16     }
17     else if (wParam == ID_BTN_EXPORT) {
18         if (!g_IsCalculated) { ShowError(hWnd, WarnNoCalc);
19             return;}
20         wstring s;
21         if (GetSaveFileNameExplorer(hWnd, s)) {
22             if (ExportDataToFile(s))
23                 g_StatusMessage = L" Отчёт сохранён!";
24             else
25                 ShowError(hWnd, ErrWrite);}
26     }
27 }

```

1.38 Процедура HandleCmdVis

```

1  /*****
2  * Процедура      : HandleCmdVis                                     *
3  *-----*
4  * Назначение     : Обработка визуализации полей                   *
5  * Входные        : hWnd - дескриптор окна, wParam - идентификатор команды *
6  * Выходные       : Нет                                             *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
9  * Иницирует рендеринг графиков через Wolfram (с проверкой готовности), *
10 * управляет состоянием отображения и выводит ошибки при сбоях.      *
11 *****/
12 void HandleCmdVis(HWND hWnd, int wParam) {
13     if (wParam == ID_BTN_VIS_A || wParam == ID_BTN_VIS_B) {
14         if (!g_IsCalculated) { ShowError(hWnd, WarnNoCalc); return; }
15         bool isA = (wParam == ID_BTN_VIS_A);
16         HBITMAP& bmp = isA ? g_hPlotBitmapA : g_hPlotBitmapB;
17         if (!bmp) { g_StatusMessage = isA ? L" Рендеринг Поля А..." :
18                     L" Рендеринг Поля Б..."; g_ActiveView = 5;
19         InvalidateRect(hWnd, NULL, TRUE); UpdateWindow(hWnd); ErrCode err;
20         if (!GenerateWolframPlot(isA, err)) {
21             g_StatusMessage = L" Ошибка рендеринга!"; g_ActiveView = 2;
22             InvalidateRect(hWnd, NULL, TRUE); UpdateWindow(hWnd); ShowError(hWnd, err);
23             return;}
24     }
25     g_StatusMessage = isA ? L" Визуализация Поля А готова!" :
26                     L" Визуализация Поля Б готова!";
27     g_ActiveView = isA ? 3 : 4;}
28 }

```

1.39 Процедура HandleWmCommand

```

1  /*****
2  * Процедура      : HandleWmCommand                                *
3  *-----*
4  * Назначение     : Диспетчеризация команд меню и кнопок          *
5  * Входные        : hWnd - окно, wParam - параметры сообщения    *
6  * Выходные       : Нет                                           *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                    *
9  * Центральный обработчик сообщений WM_COMMAND, распределяющий логику *
10 * в зависимости от нажатого элемента интерфейса.                 *
11 *****/
12 void HandleWmCommand(HWND hWnd, WPARAM wParam) {
13     int wmId = LOWORD(wParam);
14     if (wmId == ID_BTN_IMPORT || wmId == ID_BTN_SHOW)
15         HandleCmdImportShow(hWnd, wmId);
16     else if (wmId == ID_BTN_CALC || wmId == ID_BTN_EXPORT)
17         HandleCmdCalcExport(hWnd, wmId);
18     else if (wmId == ID_BTN_VIS_A || wmId == ID_BTN_VIS_B)
19         HandleCmdVis(hWnd, wmId);
20     InvalidateRect(hWnd, NULL, TRUE);
21 }

```

1.40 Функция WndProc

```

1  /*****
2  * Функция      : WndProc                                     *
3  *-----*
4  * Назначение   : Основная процедура обработки событий окна   *
5  * Входные      : hWnd, msg, wParam, lParam (стандартные Win32) *
6  * Выходные     : LRESULT - результат обработки события        *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                *
9  * Главный цикл сообщений приложения, управляет жизненным циклом окна, *
10 * отрисовкой и взаимодействием с пользователем.                *
11 *****/
12 LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam) {
13     switch (msg) { case WM_CREATE:
14         CreateInterfaceButtons(hWnd, ((LPCREATESTRUCT)lParam)->hInstance);
15         break;
16     case WM_COMMAND: HandleWmCommand(hWnd, wParam);
17         break;
18     case WM_PAINT: HandlePaint(hWnd);
19         break;
20     case WM_DESTROY: if (g_hPlotBitmapA) DeleteObject(g_hPlotBitmapA);
21         if (g_hPlotBitmapB) DeleteObject(g_hPlotBitmapB); PostQuitMessage(0);
22         break;
23     default: return DefWindowProc(hWnd, msg, wParam, lParam);}
24     return 0;
25 }
```

1.41 Функция WinMain

```

1  /*****
2  * Функция      : WinMain                                     *
3  *-----*
4  * Назначение   : Точка входа в Windows-приложение         *
5  * Входные     : hInstance, hPrevInstance, lpCmdLine, nCmdShow *
6  * Выходные    : int - код завершения программы           *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                               *
9  * Регистрирует класс окна, создает основное окно приложения и запускает *
10 * цикл обработки сообщений (Message Loop).                 *
11 *****/
12 int WINAPI WinMain(_In_ HINSTANCE hInst, _In_opt_ HINSTANCE hPInst,
13                  _In_ LPSTR cmd, _In_ int nCmdShow) {
14     WNDCLASSEXW wcex = { sizeof(WNDCLASSEX), CS_HREDRAW | CS_VREDRAW,
15                          WndProc, 0, 0, hInst, NULL, LoadCursor(NULL, IDC_ARROW),
16                          (HBRUSH)(COLOR_WINDOW + 1), NULL, L"FRC", NULL };
17     if (!RegisterClassExW(&wcex)) return 0;
18     HWND hWnd = CreateWindowW(L"FRC",
19                              L"Теория Поля - Расчет параметров векторных полей", WS_OVERLAPPEDWINDOW,
20                              CW_USEDEFAULT, CW_USEDEFAULT, 1200, 800, NULL, NULL, hInst, NULL);
21     ShowWindow(hWnd, nCmdShow); UpdateWindow(hWnd); MSG msg;
22     while (GetMessage(&msg, NULL, 0, 0)) {
23         TranslateMessage(&msg); DispatchMessage(&msg);
24     }
25     return (int)msg.wParam;
26 }

```

1.42 Функция PolyToWolframString

```

1  /*****
2  * Функция      : PolyToWolframString      *
3  *-----*
4  * Назначение   : Конвертация многочлена в формат Wolfram Language *
5  * Входные     : p - объект Polynomial    *
6  * Выходные    : wstring - строка для Mathematica/WolframScript *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:            *
9  * Преобразует внутреннее представление полинома в математическую строку, *
10 * понятную для интерпретатора Wolfram. *
11 *****/
12 wstring PolyToWolframString(const Polynomial& p) {
13     if (p.terms.empty()) return L"0"; wstringstream ss; bool first = true;
14     for (const auto& term : p.terms) { if (term.coef > 0 && !first) ss << L" + ";
15         if (term.coef < 0) ss << (first ? L"-" : L" - ");
16         ss << fixed << setprecision(4) << abs(term.coef);
17         if (term.px > 0) {
18             ss << L"*x"; if (term.px > 1) ss << L"^" << term.px;}
19         if (term.py > 0) {
20             ss << L"*y"; if (term.py > 1) ss << L"^" << term.py;}
21         if (term.pz > 0) {
22             ss << L"*z"; if (term.pz > 1) ss << L"^" << term.pz;
23         }
24         first = false;
25     }
26     return ss.str();
27 }

```

1.43 Функция GenerateWolframPlot

```

1  /*****
2  * Функция          : GenerateWolframPlot                                *
3  *-----*
4  * Назначение       : Координация процесса генерации и загрузки графика  *
5  * Входные          : isA - флаг выбора поля (true - поле A, false - поле B)*
6  *                  : err - ссылка для записи кода ошибки в случае неудачи *
7  * Выходные         : bool - true при успехе, false при возникновении ошибки*
8  *-----*
9  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                           *
10 * Получает строковое представление компонентов поля, определяет имена   *
11 * файлов скрипта и графики, вызывает функцию записи скрипта WriteScript. *
12 * Затем через RunWolfram запускает рендеринг и загружает результирующий  *
13 * BMP-файл в соответствующий глобальный дескриптор.                     *
14 *****/
15 bool GenerateWolframPlot(bool isA, ErrCode& err) {
16     auto& fmc = FieldMathCalculations;
17     wstring px = PolyToWolframString(isA ? fmc::fieldA_X : fmc::GetFieldBX());
18     wstring py = PolyToWolframString(isA ? fmc::fieldA_Y : fmc::GetFieldBY());
19     wstring pz = PolyToWolframString(isA ? fmc::fieldA_Z : fmc::GetFieldBZ());
20
21     wstring scr = isA ? L"t_a.wl" : L"t_b.wl";
22     wstring bmp = isA ? L"t_a.bmp" : L"t_b.bmp";
23     DeleteFileW(bmp.c_str());
24
25     if (!WriteScript(isA, px, py, pz, scr, bmp)) {
26         err = ErrWlScript;
27         return false;
28     }
29     if (!RunWolfram(scr)) {
30         err = ErrWlNotFound;
31         return false;
32     }
33
34     HBITMAP hBmp = (HBITMAP)LoadImageW(
35         NULL, bmp.c_str(), IMAGE_BITMAP, 0, 0, LR_LOADFROMFILE
36     );
37     if (!hBmp) {
38         err = ErrWlImage;
39         return false;
40     }
41
42     if (isA) g_hPlotBitmapA = hBmp; else g_hPlotBitmapB = hBmp;
43     return true;
44 }

```

1.44 Функция WriteScript

```

1  /*****
2  * Функция          : WriteScript                                     *
3  *-----*
4  * Назначение       : Создание файла скрипта WolframScript (.wl)    *
5  * Входные          : isA - флаг выбора поля (true - поле A, false - поле B)*
6  *                  : px, py, pz - компоненты вектора в формате Wolfram *
7  *                  : scr - путь для сохранения файла скрипта        *
8  *                  : bmp - целевой путь сохранения результирующего графика *
9  * Выходные         : bool - true, если файл успешно записан, иначе false *
10 *-----*
11 * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
12 * Рассчитывает потенциал поля, генерирует код на языке Wolfram Language *
13 * для построения комбинированного графика VectorPlot3D и ContourPlot3D, *
14 * перекодирует полученную строку в UTF-8 и записывает её в файл скрипта. *
15 *****/
16 bool WriteScript(bool isA, const wstring& px, const wstring& py,
17                  const wstring& pz, const wstring& scr, const wstring& bmp) {
18     wstring pot = isA ? L"Norm[f]" :
19         PolyToWolframString(FieldMathCalculations::GetAnalyticalPotential());
20
21     wstringstream ss;
22     ss << L"f={" << px << L"," << py << L"," << pz << L"};\npot=" << pot << L";\n"
23         << L"v=VectorPlot3D[f,{x,-2,2},{y,-2,2},{z,-2,2},VectorPoints->{7,7,7},"
24         << L"VectorColorFunction->\"Rainbow\",VectorStyle->\"Arrow\",VectorSizes->0.7,"
25         << L"VectorAspectRatio->0.1,ViewPoint->{2.5,-2.5,1.8}];\n"
26         << L"s=ContourPlot3D[pot,{x,-2,2},{y,-2,2},{z,-2,2},Contours->1,"
27         << L"ColorFunction->\"Rainbow\",ContourStyle->Directive[Opacity[0.7]],"
28         << L"Mesh->None,ViewPoint->{2.5,-2.5,1.8}];\n"
29         << L"combined=GraphicsRow[{s,v},ImageSize->1600];\n"
30         << L"Export[\"\" << bmp << L"\",combined,ImageResolution->150];\n";
31
32     wstring ws = ss.str();
33     int len = WideCharToMultiByte(CP_UTF8, 0, ws.c_str(), -1, 0, 0, 0, 0);
34     string u8(len, '\0');
35     WideCharToMultiByte(CP_UTF8, 0, ws.c_str(), -1, &u8[0], len, 0, 0);
36
37     ofstream f(scr, ios::binary);
38     if (!f.is_open()) return false;
39     f.write(u8.c_str(), len - 1);
40     return true;
41 }

```


1.45 Функция RunWolfram

```

1  /*****
2  * Функция          : RunWolfram                                     *
3  *-----*
4  * Назначение       : Исполнение сгенерированного .wl скрипта во внешней *
5  *                  : среде WolframScript                           *
6  * Входные          : scr - путь к запускаемому файлу скрипта         *
7  * Выходные         : bool - true, если процесс завершился успешно, иначе *
8  *                  : false                                           *
9  *-----*
10 * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                       *
11 * Формирует команду вызова исполняемого файла WolframScript.exe с флагом *
12 * -file. Запускает скрытый фоновый процесс без создания окна консоли,    *
13 * ожидает окончания рендеринга графики и корректно закрывает дескрипторы *
14 *****/
15 bool RunWolfram(const wstring& scr) {
16     wstring cmd = L"C:\\Program Files\\Wolfram Research"
17         L"\\WolframScript\\wolframscript.exe\\" -file " + scr;
18
19     STARTUPINFO si = { sizeof(si) };
20     si.dwFlags = STARTF_USESHOWWINDOW;
21     si.wShowWindow = SW_HIDE;
22
23     PROCESS_INFORMATION pi = { 0 };
24     if (!CreateProcessW(
25         NULL, &cmd[0], NULL, NULL, FALSE,
26         CREATE_NO_WINDOW, NULL, NULL, &si, &pi
27     )) return false;
28
29     WaitForSingleObject(pi.hProcess, INFINITE);
30     CloseHandle(pi.hProcess);
31     CloseHandle(pi.hThread);
32     return true;
33 }

```

1.46 Процедура DrawVisualization

```

1  /*****
2  * Процедура      : DrawVisualization      *
3  *-----*
4  * Назначение     : Отрисовка графического представления выбранного поля *
5  * Входные       : hdc - дескриптор контекста устройства вывода      *
6  *               : rect - прямоугольная область для рисования        *
7  * Выходные      : Нет                                                *
8  *-----*
9  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:      *
10 * Определяет активную битовую карту поля, рассчитывает пропорциональные *
11 * координаты вывода с помощью GetDrawCoords, масштабирует изображение *
12 * (StretchBlt с качественным сглаживанием) и выводит подписи к графикам.*
13 *****/
14 void DrawVisualization(HDC hdc, RECT rect) {
15     HBITMAP hBmp = (g_ActiveView == 3) ? g_hPlotBitmapA : g_hPlotBitmapB;
16     if (!hBmp) return;
17
18     HDC hdcMem = CreateCompatibleDC(hdc);
19     HBITMAP hOldBmp = (HBITMAP)SelectObject(hdcMem, hBmp);
20
21     BITMAP bitmap;
22     GetObject(hBmp, sizeof(BITMAP), &bitmap);
23
24     int lblH = 35, dW, dH, dX, dY;
25     GetDrawCoords(bitmap, rect, lblH, dW, dH, dX, dY);
26
27     SetStretchBltMode(hdc, HALFTONE);
28     StretchBlt(
29         hdc, dX, dY, dW, dH, hdcMem, 0, 0,
30         bitmap.bmWidth, bitmap.bmHeight, SRCCOPY
31     );
32
33     SelectObject(hdcMem, hOldBmp);
34     DeleteDC(hdcMem);
35
36     DrawLabels(hdc, dX, dW, dY, dH, lblH);
37 }

```

1.47 Процедура GetDrawCoords

```

1  /*****
2  * Процедура      : GetDrawCoords *
3  *-----*
4  * Назначение     : Расчет координат вывода с сохранением пропорций *
5  * Входные        : bmp - структура с характеристиками изображения *
6  *                : r   - прямоугольник доступной области рисования *
7  *                : lblH - высота области, резервируемой под подписи *
8  * Выходные       : dW, dH - рассчитанные ширина и высота для вывода *
9  *                : dX, dY - рассчитанные координаты левого верхнего угла *
10 *-----*
11 * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ: *
12 * Сравнивает соотношение сторон исходного изображения и области вывода *
13 * (с вычетом места под текст). Центрирует изображение по горизонтали *
14 * или вертикали в зависимости от того, по какой оси произошло сжатие. *
15 *****/
16 void GetDrawCoords(const BITMAP& bmp, const RECT& r, int lblH,
17                   int& dW, int& dH, int& dX, int& dY) {
18     int rW = r.right - r.left;
19     int rH = r.bottom - r.top - lblH;
20     if (rH < 50) rH = 50;
21
22     double aspB = (double)bmp.bmWidth / bmp.bmHeight;
23     double aspR = (double)rW / rH;
24
25     if (aspB > aspR) {
26         dW = rW;
27         dH = (int)(rW / aspB);
28         dX = r.left;
29         dY = r.top + (rH - dH) / 2;
30     } else {
31         dW = (int)(rH * aspB);
32         dH = rH;
33         dX = r.left + (rW - dW) / 2;
34         dY = r.top;
35     }
36 }

```

1.48 Процедура DrawLabels

```

1  /*****
2  * Процедура      : DrawLabels                                     *
3  *-----*
4  * Назначение     : Отрисовка текстовых подписей под графиками поля *
5  * Входные       : hdc - контекст устройства для рисования текста *
6  *               : dX, dW, dY, dH - геометрия выведенного графика *
7  *               : lblH - высота, выделенная под отрисовку текста *
8  * Выходные      : Нет                                           *
9  *-----*
10 * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                     *
11 * Инициализирует шрифт Segoe UI Bold, настраивает прозрачный фон текста *
12 * и выводит симметричные подписи (для эквипотенциалей слева и для *
13 * силовых линий справа) под отмасштабированным изображением.      *
14 *****/
15 void DrawLabels(HDC hdc, int dX, int dW, int dY, int dH, int lblH) {
16     bool isA = (g_ActiveView == 3);
17     wstring lL = isA ? L"Поле А: эквипотенциальные поверхности"
18                   : L"Поле В: эквипотенциальные поверхности";
19     wstring lR = isA ? L"Поле А: силовые линии" : L"Поле В: силовые линии";
20
21     HFONT hF = CreateFontW(
22         16, 0, 0, 0, FW_BOLD, 0, 0, 0, DEFAULT_CHARSET,
23         0, 0, CLEARTYPE_QUALITY, 0, L"Segoe UI"
24     );
25     HFONT hOldF = (HFONT)SelectObject(hdc, hF);
26
27     int oldBk = SetBkMode(hdc, TRANSPARENT);
28     COLORREF oldCol = SetTextColor(hdc, RGB(50, 50, 50));
29     int lY = dY + dH + 5;
30
31     RECT rL = { dX, lY, dX + dW / 2, lY + lblH };
32     RECT rR = { dX + dW / 2, lY, dX + dW, lY + lblH };
33
34     DrawTextW(hdc, lL.c_str(), -1, &rL, DT_CENTER | DT_TOP | DT_SINGLELINE);
35     DrawTextW(hdc, lR.c_str(), -1, &rR, DT_CENTER | DT_TOP | DT_SINGLELINE);
36
37     SetTextColor(hdc, oldCol);
38     SetBkMode(hdc, oldBk);
39     SelectObject(hdc, hOldF);
40     DeleteObject(hF);
41 }

```

1.49 Процедура ShowError

```

1  /*****
2  * Процедура      : ShowError                                     *
3  *-----*
4  * Назначение     : Централизованный вывод сообщений об ошибках *
5  * Входные        : hWnd - окно, code - код ошибки              *
6  * Выходные       : Нет                                          *
7  *-----*
8  * ОПИСАНИЕ ФУНКЦИОНИРОВАНИЯ:                                   *
9  * Отображает MessageBox с соответствующим заголовком и текстом ошибки *
10 * на основе переданного кода ошибки (enum).                      *
11 *****/
12 void ShowError(HWND hWnd, ErrCode code) {
13     const wchar_t* m = L"Неизвестная ошибка"; const wchar_t* t = L"Ошибка";
14     UINT ic = MB_ICONERROR;
15     if (code == ErrParse) { m = L"Файл поврежден, пуст или имеет неверный формат!";
16         t = L"Ошибка парсинга"; }
17     else if (code == WarnNoData) { m = L"Сперва необходимо загрузить данные!";
18         t = L"Предупреждение"; ic = MB_ICONWARNING; }
19     else if (code == WarnNoCalc) { m = L"Сперва необходимо провести расчеты!";
20         t = L"Предупреждение"; ic = MB_ICONWARNING; }
21     else if (code == ErrWrite) { m = L"Ошибка записи файла!"; }
22     else if (code == ErrWlScript) {
23         m = L"Не удалось создать файл скрипта (.wl)! Проверьте права.";
24         t = L"Ошибка визуализации"; }
25     else if (code == ErrWlNotFound) {
26         m = L"WolframScript не найден!\nПроверьте путь к wolframscript.exe.";
27         t = L"Ошибка визуализации"; }
28     else if (code == ErrWlImage) {
29         m = L"Изображение не создано!\nВозможно, скрипт завершился с ошибкой.";
30         t = L"Ошибка визуализации"; }
31     MessageBoxW(hWnd, m, t, ic | MB_OK);
32 }

```

[illegible]